



24. Linear Discriminant Analysis

Finding the Clear Line Between Classes

Previous Section:

 [23. Singular Value Decomposition](#)

Contents:

[1. What is Discriminant Analysis?](#)

[2. Linear Discriminant Analysis – The Overview](#)

[3. Linear Discriminant Analysis with One Feature](#)

[4. Linear Discriminant Analysis: Adding Another Feature](#)

[5. Linear Discriminant Analysis: The Fisher Steps](#)

[6. Linear Discriminant Analysis with Python](#)

[Summary](#)

[References](#)

This section is about Linear Discriminant Analysis (or just LDA). Now at first glance, LDA might feel a bit confusing. Because if you think about it: ***Isn't every classification model trying to draw a boundary?***

- Decision Trees split the space
- Support Vector Machines draw margins
- Even Logistic Regression draws a line

→ So what makes LDA different?

Here's the thing. Most models are trying to **separate data**. LDA is trying to **understand why the data is separable in the first place**.

Think about it like this. Imagine you have two classes: Not just points scattered randomly. But groups with structure, with shape, with direction.

Most models will say: *"Let me draw a boundary that separates them as best as possible."*

But LDA takes a step back and asks:

"What is the best direction to look at this data... so that the classes become the most distinguishable?"

And that's a very different mindset. Instead of drawing a complex boundary in a high-dimensional space. LDA does something surprisingly elegant:

- It **projects the data onto a new axis**. One where the classes are **as far apart as possible**. While keeping each class as **tight as possible**

So it's not just separating. It's **reframing the problem**.

And here's where it gets even more interesting. Because the moment you say: ***"Let's project data onto a lower-dimensional space..."*** → You're no longer just doing classification. You're also doing **dimensionality reduction**.

But unlike PCA (*which you've probably seen before, though it is covered after LDA and QDA*): PCA cares about **variance** → LDA cares about **class separation**

- So if PCA says: *"Let me preserve as much information as possible..."*
- LDA says: *"Let me preserve what actually helps us tell classes apart."*

That's why LDA feels different. It's not trying to be flexible like trees. Not trying to maximize margins like SVM. It's trying to find something much more deliberate: **A clear, meaningful line between classes**.

And once you see that, LDA stops being just another model. It becomes a way of thinking:

"Don't just separate the data... understand the direction that makes separation obvious."

1. What is Discriminant Analysis?

Alright, let's slow down for a moment and start from the core idea. Discriminant Analysis sounds like one of those terms that tries a bit too hard to sound complicated. But if you break it down, "Discriminant" simply means **to distinguish**, "Analysis" means **to study or figure something out**.

So together? It's about **figuring out how to distinguish between different groups of data**

Now let's make this real. Imagine you are given a dataset of students. Each student has features like: Study hours per week; Attendance rate; Assignment scores → And each student is labeled as "Pass" or "Fail"

At first, it might look like just a bunch of numbers. But hidden inside that data, there's a pattern. Somewhere.

Maybe:

- Students who study more and attend regularly tend to pass
- Students with low attendance and low scores tend to fail

But here's the challenge: The data is not perfectly clean. There is overlap. Some students break the pattern.

So now the question becomes: *Can we draw a boundary that separates "Pass" and "Fail" students as clearly as possible?* That's exactly where **Discriminant Analysis** comes in.

Discriminant Analysis is a family of techniques used in **supervised learning**—which means we already know the correct labels (Pass / Fail) → And the goal is very specific: **Find a decision boundary that best separates the classes**

But here's the important part. It doesn't just randomly draw a boundary. It tries to find a boundary that is: **Meaningful. Based on the structure of the data. Optimized for class separation.**

Let's go back to our student example. Imagine plotting the students on a graph:

- X-axis → Study hours
- Y-axis → Attendance

Now you'll see two clusters forming:

- One cluster (Pass students)
- One cluster (Fail students)

Discriminant Analysis looks at this and asks: *"Where should I draw the line... so that these two groups are as separated as possible?"*

But it goes even deeper than that. Instead of just drawing a line. It tries to understand:

┆ *"What combination of features best separates these students?"*

Maybe it's not just: Study hours alone or Attendance alone

But something like: $0.7 \times \text{StudyHours} + 0.3 \times \text{Attendance}$

This new combination becomes a **new axis**. And along this axis? Pass and Fail students become much easier to distinguish.

That's the essence of Discriminant Analysis. Not just separating data. But **transforming the perspective so that separation becomes clear**.

Now let's contrast this with something you already know. Component Analysis methods like PCA:

- Do **not use labels** (unsupervised learning)
- Focus on **capturing variance**
- Try to preserve as much information as possible

So PCA might say: *"Let me find directions where the data spreads out the most."*

But Discriminant Analysis has a completely different goal. It says: *"I don't care about overall spread. I care about separating classes."*

So instead of maximizing variance. It tries to:

- **Maximize the distance between classes**
- **Minimize the spread within each class**

Going back to the student example:

- PCA might group students based on general variability
- Discriminant Analysis will group them based on **Pass vs Fail separation**

And that difference? It's everything. So in one sentence:

Discriminant Analysis is a supervised learning approach that finds the best way to separate labeled groups by understanding the structure that distinguishes them.

And once you see it that way. It stops being just a definition. It becomes a very intuitive idea: *"If you want to classify... first understand what truly makes things different."*



Important Note. About this relation:

$$0.7 \times \text{StudyHours} + 0.3 \times \text{Attendance}$$

At first glance, this looks like a line. But it isn't. Because something is missing. There's no "=" sign. No threshold. No boundary. Right now, it's just a direction. A way of combining features. A way of looking at the data. Only when we say something like...

$$0.7 \times \textit{StudyHours} + 0.3 \times \textit{Attendance} = c$$

does it become a line. And only when we compare...

$$0.7 \times \textit{StudyHours} + 0.3 \times \textit{Attendance} > c$$

does it become a decision boundary.

So this expression alone? Not a line. Just the lens. The line comes later.

2. Linear Discriminant Analysis – The Overview

Now that we understand what Discriminant Analysis is trying to do. Let's talk about its most iconic form: **Linear Discriminant Analysis (LDA)**

At first, the name sounds intimidating. But if you strip it down:

- "Linear" → we are working with straight lines (or linear combinations)
- "Discriminant" → we want to separate classes
- "Analysis" → we are figuring out how to do it

So LDA is simply: **Finding the best linear way to separate different classes**

But here's where it gets interesting. LDA is not just a classification method. It's also a **dimensionality reduction technique**. And unlike PCA, it has a very clear goal:

┆ *"Reduce dimensions... but never lose the ability to separate classes."*

Let's go back to something familiar. Our student dataset: Study hours; Attendance; Assignment scores; Final label → Pass / Fail. Now imagine this dataset has 20 features instead of 3. Suddenly, everything becomes messy. You can't visualize it. You can't easily see the pattern. Most models will try to handle this complexity directly.

But LDA does something smarter. It says:

- ***"What if I compress all these features... into a smaller number of dimensions..."***
- *"But in a way that keeps Pass and Fail students clearly separated?"*

So instead of keeping all 20 features. LDA creates **new features**. Not random ones. But **linear combinations** like:

$$0.5 \times \textit{StudyHours} + 0.3 \times \textit{Attendance} + 0.2 \times \textit{AssignmentScore}$$

These combinations form new axes. And along these axes? The separation between classes becomes much clearer.

Now here's the core philosophy of LDA. And this is the part you really want to remember. LDA is trying to balance two things:

- **Between-class variance (push classes apart)** → Make Pass and Fail students as far from each other as possible
- **Within-class variance (keep each class tight)** → Keep Pass students close to each other and keep Fail students close to each other

So the goal becomes: **Maximize the distance between classes, and minimize the spread within each class**

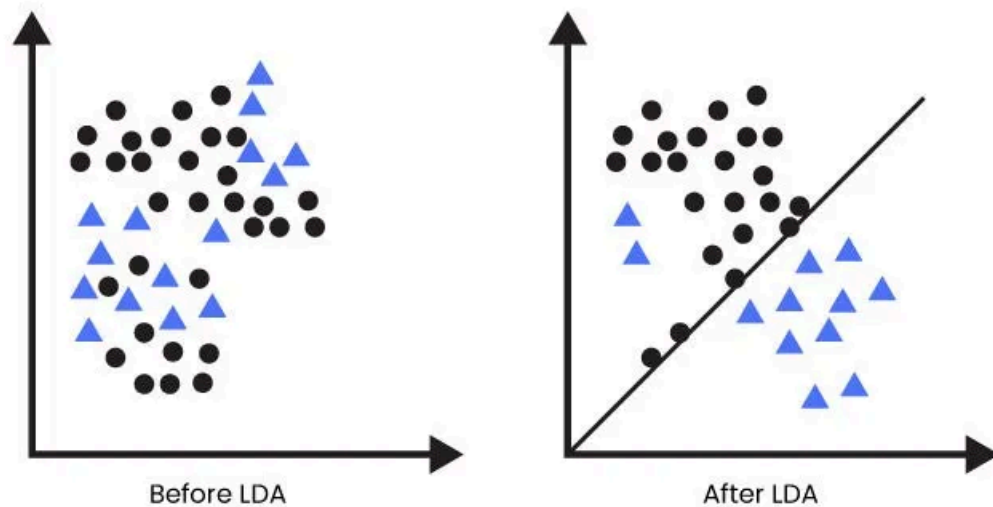
If you think about it: That's exactly what a "perfect" classifier would want. Clear separation. Minimal overlap.

Now compare this with other models you already know:

- Decision Trees → split data step by step
- Random Forests → combine multiple trees
- Logistic Regression → draw a decision boundary
- SVM → maximize the margin

They all try to classify. But LDA? It takes a different route.

(The following image is used as reference only)



It first asks: *"Can I transform the data into a space where classification becomes easy?"*. And only then: *"Now... separating them is trivial."*

That's why LDA feels elegant. It doesn't fight the data. It **rearranges the data** so the answer becomes obvious. And this is what makes LDA special. It serves **two purposes at the same time**:

- A **classifier** → because it creates a decision boundary
- A **dimensionality reducer** → because it compresses features

This becomes extremely powerful when working with high-dimensional data. Because sometimes, keeping all the information is not the goal → Keeping the **right information** is.

- So while PCA says: *"Let me preserve as much variance as possible..."* → LDA says: *"Let me preserve what actually helps separate classes."*

And once you understand that, you'll see LDA not just as a model. But as a strategy:

"Don't just reduce dimensions... reduce them in a way that makes the problem easier to solve."

3. Linear Discriminant Analysis with One Feature

Enough theory for this time. Let's see LDA in action. And to make things *crystal clear*... We're going to use the simplest setup possible: **Only one feature**

The Setup - Imagine we are studying students again. But this time, we only look at **one thing**: Their **School Year** (1st year or 2nd year). And we want to predict: Do

they have a **part-time job** (Yes / No)?

The following dataset is given"

StudentID	School Year	Part-Time Job
001	1	No
002	2	Yes
003	2	Yes
004	2	No
005	1	No
006	1	No
007	2	Yes
008	1	No
009	1	No
010	1	Yes

At first glance, this looks *too simple*. Just 1s and 2s. But don't underestimate it. Because even here, there's still a pattern hiding.

The goal: We want a rule like: **"If School Year is ___ → predict Yes or No"**

Since this is **1D (one dimension)**, LDA will not draw a line. It will draw a **threshold**.

Step 1: Group Data by Class - Let's separate the students.

- **Class "Yes" (Has part-time job)**
 - Students: 002, 003, 007, 010
 - Values: [2, 2, 2, 1]
 - Mean: $\mu_1 = \frac{2+2+2+1}{4} = 1.75$
- **Class "No" (No part-time job)**
 - Students: 001, 004, 005, 006, 008, 009
 - Values: [1, 2, 1, 1, 1, 1]
 - Mean: $\mu_2 = \frac{1+2+1+1+1+1}{6} \approx 1.17$

Already, we see something interesting:

- Students with jobs tend to be closer to **year 2**
- Students without jobs cluster near **year 1**

Step 2: Measure the Spread (Variance) - Now LDA asks: "How spread out is each class?"

- **Variance of "Yes" class:** $s_1^2 = 0.125$
- **Variance of "No" class:** $s_2^2 \approx 0.154$

But here's a key assumption in LDA: **All classes share the same variance.** So we combine them into a **pooled variance**:

$$s^2 = \frac{4 \cdot 0.125 + 5 \cdot 0.154}{4 + 5} \approx 0.141$$

→ This represents the **overall spread** of the data.

Step 3: Find the Decision Boundary

Now comes the elegant part. In 1D, LDA places the boundary **between the two means**.

$$x^* = \frac{\mu_1 + \mu_2}{2} = \frac{1.75 + 1.17}{2} = 1.46$$

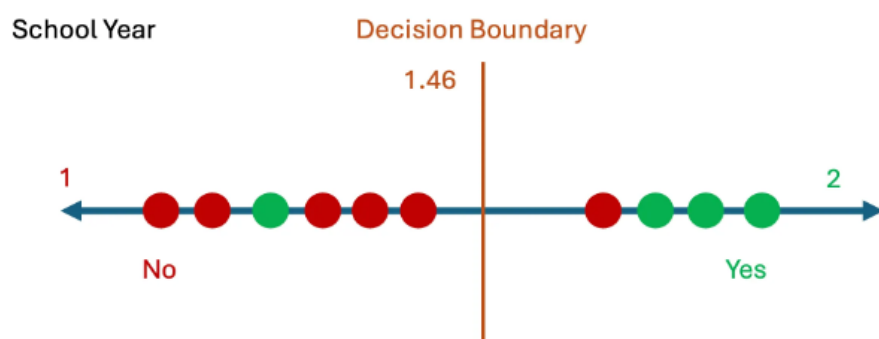
That's it. That's the model. No trees. No complex equations. Just one number: 1.46

Step 4: Interpretation

Now we turn it into a rule:

- If $SchoolYear \geq 1.46 \rightarrow$ predict *Yes*
- If $SchoolYear < 1.46 \rightarrow$ predict *No*

Visually, think of a number line (Points representing students are placed at **School Year = 1** or **2**):



- Year 1 → left side → "No"
- Year 2 → right side → "Yes"

Final Predictions

StudentID	School Year	Actual	Predicted
001	1	No	No
002	2	Yes	Yes
003	2	Yes	Yes
004	2	No	Yes → Incorrect
005	1	No	No
006	1	No	No
007	2	Yes	Yes
008	1	No	No
009	1	No	No
010	1	Yes	No → Incorrect

Result: 2 mistakes → 8 correct predictions → **Accuracy = 80%**

This example might look simple. But it captures the *essence* of LDA. LDA didn't:

- Memorize the data
- Try random splits
- Overcomplicate the problem

Instead, it did something very intentional:

- It looked at **class centers (means)**
- Considered their **spread (variance)**
- And placed a boundary where separation is most reasonable

Even with just one feature, LDA already shows its philosophy: *"Find the direction (or point)... where classes become the most distinguishable."*

And once you understand this in 1D. Moving to higher dimensions is just the same idea, just with more directions to choose from.

4. Linear Discriminant Analysis: Adding Another Feature

Alright, now things get more interesting. So far, we only used **one feature**: *SchoolYear*. And LDA gave us a simple threshold. Clean. Easy. Almost *too easy*.

Now let's add another feature: *EnglishLevel* (1–5)

StudentID	School Year	English Level	Part-Time Job
001	1	5	No
002	2	1	Yes
003	2	2	Yes
004	2	3	No
005	1	3	No
006	1	2	No
007	2	3	Yes
008	1	4	No
009	1	1	No
010	1	1	Yes

Now we have **two features**: *SchoolYear* and *EnglishLevel*

And this changes everything. Because we are no longer in 1D. We are now in **2D space**.

What Changes? Before this, LDA gave us a **single threshold** → Now LDA gives us a **line**

Instead of saying: "**Left = No, Right = Yes**" → We now say: "**Above the line = Yes**"; "**Below the line = No**"

But here's the key question: *How does LDA decide where to place that line?*

Core Idea of LDA in 2D - LDA does something very deliberate. It doesn't directly draw the line. Instead, it first finds a **new direction**. A direction defined by weights: w_1 (for *SchoolYear*); w_2 (for *EnglishLevel*)

This direction is used to **project the data**. And along this projection? The classes should be as separable as possible.

The objective is captured by this function:

$$f(w_1, w_2) = \frac{(\mu_1 - \mu_2)^2}{s_1^2 + s_2^2}$$

This might look intimidating. But it's actually very intuitive.

- The **numerator** → how far apart the classes are
- The **denominator** → how spread out they are

So LDA is trying to:

- **Maximize separation between classes**
- **Minimize spread within classes**

In other words: *"Push the classes apart... while keeping each one tight"*

Applying LDA to the Dataset (2D Case) - Now let's actually compute things.

Step 1: Class Means

- **Class "Yes" (Has Part-Time Job)**
 - Data points: $(2, 1), (2, 2), (2, 3), (1, 1)$
 - Mean vector: $\mu_1 = \left[\frac{2+2+2+1}{4}, \frac{1+2+3+1}{4} \right] = [1.75, 1.75]$
- **Class "No" (No Part-Time Job)**
 - Data points: $(1, 5), (2, 3), (1, 3), (1, 2), (1, 4), (1, 1)$
 - Mean vector: $\mu_2 = \left[\frac{1+2+1+1+1+1}{6}, \frac{5+3+3+2+4+1}{6} \right] = [1.17, 3.00]$

Already, we see something interesting:

- "Yes" students → lower English level on average
- "No" students → higher English level

Step 2: Mean Difference

Now we compare the two classes:

$$(\mu_1 - \mu_2)^2 = ([1.75, 1.75] - [1.17, 3.00])^2 = [0.3364, 1.5625]$$

This tells us: The difference in **English Level** is much larger than School Year

Step 3: Within-Class Variance

Now we measure how spread out each class is.

- **Within-Class Variance of Class "Yes":** $\mu_1 = [1.75, 1.75]$

(School, English)	Deviation ($\mathbf{x}_i - \mu_1$)	Squared Distance
(2, 1)	(0.25, -0.75)	$0.25^2 + (-0.75)^2 = 0.625$
(2, 2)	(0.25, 0.25)	$0.25^2 + 0.25^2 = 0.125$

(2, 3)	(0.25, 1.25)	$0.25^2 + 1.25^2 = 1.625$
(1, 1)	(-0.75, -0.75)	$0.75^2 + 0.75^2 = 1.125$

→ Total variance: $s_1^2 = 0.625 + 0.125 + 1.625 + 1.125 = 3.50$

- **Within-Class Variance of Class "No":** $\mu_2 = [1.17, 3.00]$

(School, English)	Deviation ($x_i - \mu_2$)	Squared Distance
(1, 5)	(-0.17, 2.00)	$(-0.17)^2 + 2^2 = 4.0289$
(2, 3)	(0.83, 0.00)	$(0.83^2) + 0 = 0.6889$
(1, 3)	(-0.17, 0.00)	$(-0.17)^2 + 0^2 = 0.0289$
(1, 2)	(-0.17, -1.00)	$(-0.17)^2 + (-1)^2 = 1.0289$
(1, 4)	(-0.17, 1.00)	$(-0.17)^2 + 1^2 = 1.0289$
(1, 1)	(-0.17, -2.00)	$(-0.17)^2 + (-2)^2 = 4.0289$

→ Total variance: $s_2^2 = 4.0289 + 0.6889 + 0.0289 + 1.0289 + 1.0289 + 4.0289 = 10.8344$

Notice something: Class "No" is much more spread out

Step 4: LDA Ratio

Now we combine everything:

$$f(w_1, w_2) = \frac{[0.3364, 1.5625]}{3.50 + 10.8344} = \frac{1}{14.3344} [0.3364, 1.5625] = [0.02347, 0.10899]$$

This is the important part. Look at the values:

- School Year → 0.02347
- English Level → 0.10899

English Level contributes **much more** to class separation. So LDA is basically telling us:

"If you want to distinguish students with part-time jobs... English Level matters more than School Year."

And this is powerful. Because LDA doesn't just classify. It also gives you **insight**. It tells you: Which features actually matter

At this point, we haven't even drawn the line yet. But we already know:

- The direction of separation
- The importance of each feature
- The structure of the data

And that's very "machine learning" thinking. Observe first. Understand structure. Then act.

Now that we understand class means, variance, feature importance. We are ready for the final step: Constructing the actual decision boundary.

5. Linear Discriminant Analysis: The Fisher Steps

Alright... now we reach the part where LDA stops being "just an idea"... and becomes a **method**. A very structured one.

Up until now, we've been thinking intuitively:

Push classes apart → Keep them tight → Find a good direction

But Fisher's method turns that intuition into **steps you can actually compute**.

The Fisher Steps (Big Picture) - LDA doesn't randomly draw a line. It follows a sequence:

1. Find where each class lives (**means**)
2. Measure how messy each class is (**scatter**)
3. Find the best direction (**projection vector**)
4. Project everything onto that direction
5. Decide a threshold

And from that, the decision boundary appears.

Finding the LDA Decision Boundary (2D Line) - We now continue from our 2D example:

- Features: School Year, English Level
- Classes: Yes / No

Step 1 — Class Means (Already computed, but now we *use* them)

$\mu_1 = [1.75, 1.75]$, $\mu_2 = [1.17, 3.00]$ → Mean difference: $\mu_1 - \mu_2 = [0.58, -1.25]$

This vector already hints: Move right (**School Year ↑**) and move down (**English Level ↓**)

Step 2 — Within-Class Scatter Matrix

Now we measure how spread out each class is. But instead of simple variance, we use matrices.

The general formula:

$$S_w = \sum_{x \in C_1} (x - \mu_1)(x - \mu_1)^\top + \sum_{x \in C_2} (x - \mu_2)(x - \mu_2)^\top$$

Or simply in this case: $S_w = S_1 + S_2$

• Class "Yes" Scatter Matrix

(School, English)	Deviation ($\mathbf{x}_i - \boldsymbol{\mu}_1$)	Outer Product ($(\mathbf{x}_i - \boldsymbol{\mu}_1)(\mathbf{x}_i - \boldsymbol{\mu}_1)^\top$)
(2, 1)	(0.25, -0.75)	$\begin{bmatrix} 0.0625 & -0.1875 \\ 0.1875 & 0.5625 \end{bmatrix}$
(2, 2)	(0.25, 0.25)	$\begin{bmatrix} 0.0625 & 0.0625 \\ 0.0625 & 0.0625 \end{bmatrix}$
(2, 3)	(0.25, 1.25)	$\begin{bmatrix} 0.0625 & 0.3125 \\ 0.3125 & 1.5625 \end{bmatrix}$
(1, 1)	(-0.75, -0.75)	$\begin{bmatrix} 0.5625 & 0.5625 \\ 0.5625 & 0.5625 \end{bmatrix}$

After summing all outer products: $S_1 = \begin{bmatrix} 0.75 & 1.125 \\ 1.125 & 2.75 \end{bmatrix}$

• Class "No" Scatter Matrix

(School, English)	Deviation ($\mathbf{x}_i - \boldsymbol{\mu}_2$)	Outer Product ($(\mathbf{x}_i - \boldsymbol{\mu}_2)(\mathbf{x}_i - \boldsymbol{\mu}_2)^\top$)
(1, 5)	(-0.17, 2.00)	$\begin{bmatrix} 0.0289 & -0.34 \\ -0.34 & 4 \end{bmatrix}$
(2, 3)	(0.83, 0.00)	$\begin{bmatrix} 0.6889 & 0 \\ 0 & 0 \end{bmatrix}$
(1, 3)	(-0.17, 0.00)	$\begin{bmatrix} 0.0289 & 0 \\ 0 & 0 \end{bmatrix}$

(1, 2)	(-0.17, -1.00)	$\begin{bmatrix} 0.0289 & 0.17 \\ 0.17 & 1 \end{bmatrix}$
(1, 4)	(-0.17, 1.00)	$\begin{bmatrix} 0.0289 & -0.17 \\ -0.17 & 1 \end{bmatrix}$
(1, 1)	(-0.17, -2.00)	$\begin{bmatrix} 0.0289 & 0.34 \\ 0.34 & 1 \end{bmatrix}$

After summing all outer products: $S_2 = \begin{bmatrix} 0.8333 & 0 \\ 0 & 10 \end{bmatrix}$

Combined Scatter Matrix: $S_w = S_1 + S_2 = \begin{bmatrix} 1.5833 & 1.125 \\ 1.125 & 12.75 \end{bmatrix}$

This matrix represents how messy the data is **overall**

Step 3 — Optimal Projection Direction

Now comes the core formula:

$$w = S_w^{-1}(\mu_1 - \mu_2)$$

Inverse of scatter matrix: $S_w^{-1} = \begin{bmatrix} 0.725 & -0.1978 \\ -0.1978 & 0.4176 \end{bmatrix}$

Multiply: $w = S_w^{-1} = \begin{bmatrix} 0.725 & -0.1978 \\ -0.1978 & 0.4176 \end{bmatrix} \begin{bmatrix} 0.58 \\ -1.25 \end{bmatrix} = \begin{bmatrix} 0.6679 \\ -0.6367 \end{bmatrix}$

Transpose: $w^T = [0.6679 \quad -0.6367]$

This vector is everything. It tells us: **School Year** → **positive influence**; **English Level** → **negative influence**

In simple terms: **Higher year** → **more likely "Yes"**; **Higher English** → **more likely "No"**

Step 4 — Project Means & Find Threshold

Project both class centers:

$$m_1 = w^T \mu_1 = [0.6679 \quad -0.6367] \begin{bmatrix} 1.75 \\ 1.75 \end{bmatrix} = 0.0546$$

$$m_2 = w^T \mu_2 = [0.6679 \quad -0.6367] \begin{bmatrix} 1.17 \\ 3.00 \end{bmatrix} = -1.12865$$

$$\text{Threshold: } t = \frac{m_1 + m_2}{2} = \frac{0.0546 + (-1.12865)}{2} = -0.537025$$

Step 5 — Decision Rule

For any student: Predict Yes if $w^T x \geq t$ (-0.536025)

Final Decision Boundary

Now we bring everything back to 2D. The boundary is:

$$w^T x = 0.6679 \cdot (\text{School Year}) - 0.6367 \cdot (\text{English Level}) = -0.537025$$

Rewriting: $\text{English Level} \approx 1.049 \cdot (\text{School Year}) + 0.844$

This is the **line**. **Above** → **one class**; **Below** → **the other**

Final Predictions

School Year	English	$w^T x$	Predicted	Actual
1	5	-2.5156	No	No
2	1	0.6991	Yes	Yes
2	2	0.0624	Yes	Yes
2	3	-0.5743	No	No
1	3	-1.2422	No	No
1	2	-0.6055	No	No
2	3	-0.5743	No	Yes → Incorrect
1	4	-1.8789	No	No
1	1	0.0312	Yes	No → Incorrect
1	1	0.0312	Yes	Yes

Final Result: 2 misclassifications → Accuracy = **80%**

What This Really Shows: This is the full LDA story:

From raw data → to means → to scatter → to projection → to decision boundary

And notice something subtle. LDA never tried to fit complex curves or memorize every point. Instead, it asked: *“What is the best perspective where separation becomes simple?”* That’s the Fisher philosophy. Not brute force. But clarity.

And once you see it, that line in 2D? It’s not just a boundary. It’s the **result of understanding the structure of the data**.

6. Linear Discriminant Analysis with Python

Linear Discriminant Analysis (or LDA) is straightforward using the Scikit-Learn. Using a really “toy-ish” student dataset:

```
import pandas as pd

data = {"StudentID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        "School_Year": [1, 2, 2, 2, 1, 1, 2, 1, 1, 1],
        "Gpa_Score": [3.5, 2.7, 2.4, 3.0, 3.5, 3.3, 2.1, 2.9, 1.8, 2.4],
        "Fitness_Score": [9.5, 8.4, 8.5, 8.8, 8.3, 7.8, 7.8, 8.5, 9.6, 9.4],
        "English_Level": [5, 1, 2, 3, 3, 2, 3, 4, 1, 1],
        "Major": ["IT", "IT", "Business", "Marketing", "Business", "IT",
                  "Marketing", "IT", "Business", "Marketing"],
        "Part_Time_Job": ["No", "Yes", "Yes", "No", "No", "No", "Yes", "No",
                          "No", "Yes"]}

df = pd.DataFrame(data).set_index('StudentID')
# Replace the Yes No in Part Time Job
df['Part_Time_Job'] = df['Part_Time_Job'].map({'Yes': 1, 'No': 0})
# One-Hot Encoding on Major
df = pd.get_dummies(df, columns=['Major']).replace({True: 1, False: 0})

from sklearn.model_selection import train_test_split

# Features and Target
target = 'Part_Time_Job'
features = df.columns.drop(target)
# 80% training and 20% test
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
```

```
y_train = train_df[target]
y_test = test_df[target]
```

Here, we implemented two models:

- Model 1: LDA using only the School Year

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score

# Model 1: Only School Year
features1 = ['School Year']
X_train1 = train_df[features1]
X_test1 = test_df[features1]
model1 = LinearDiscriminantAnalysis(solver='svd')
model1.fit(X_train1, y_train)
t = (model1.means_[0] + model1.means_[1]) / 2
print(f"Model 1 Decision boundary (School Year threshold): x
= {t[0]:.2f}")
y_pred1 = model1.predict(X_test1)
accuracy1 = accuracy_score(y_test, y_pred1)
print("Accuracy:", accuracy1)
```

Model 1 Decision boundary (School Year threshold): x = 1.43

Accuracy: 1.0

- Model 2: LDA using only School Year + English Level

```

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score

# Model 2: School Year + English Level
features2 = ['School Year', 'English_Level']
X_train2 = train_df[features2]
X_test2 = test_df[features2]
model2 = LinearDiscriminantAnalysis(solver='svd')
model2.fit(X_train2, y_train)
coef = model2.coef_[0]
intercept = model2.intercept_[0]
slope = -coef[0]/coef[1]
intercept_y = -intercept/coef[1]
print(f"Model 2 (2D) Decision Boundary: English_Level = {slope:.3f}*School_Year + {intercept_y:.3f}")
print(f"Model 2 Threshold (projected mean between classes on LDA axis): {intercept_y:.3f}")
y_pred2 = model2.predict(X_test2)
accuracy2 = accuracy_score(y_test, y_pred2)
print("Accuracy:", accuracy2)

```

Model 2 (2D) Decision Boundary: English_Level = 1.800*School_Year + -0.252

Model 2 Threshold (projected mean between classes on LDA axis): -0.252

Accuracy: 0.5

- Plot the decision boundary for both models

```

import matplotlib.pyplot as plt
import numpy as np

# Model 1: 1D LDA (School Year)
plt.figure(figsize=(12,5))
# 1D subplot
plt.subplot(1,2,1)
# Plot train points
for label in [0,1]:
    plt.scatter(train_df['School Year'][y_train==label],
                np.zeros_like(train_df['School Year'][y_train==label]) +

```

```

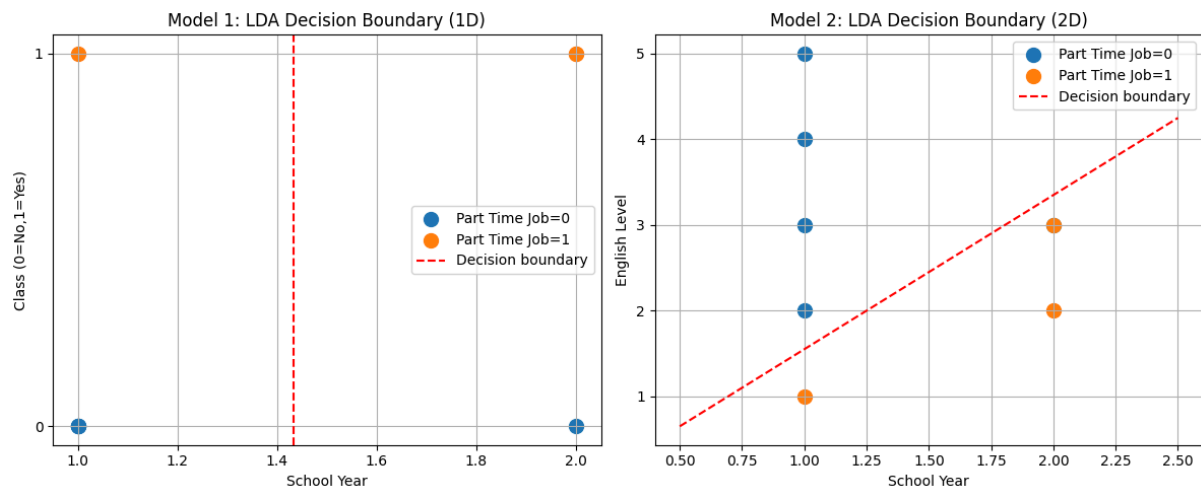
        label, label=f'Part Time Job={label}', s=100)

# Decision threshold
threshold1 = (model1.means_[0] + model1.means_[1]) / 2
plt.axvline(x=threshold1, color='red', linestyle='--', label
='Decision boundary')
plt.xlabel('School Year')
plt.ylabel('Class (0=No,1=Yes)')
plt.title('Model 1: LDA Decision Boundary (1D)')
plt.legend()
plt.yticks([0,1])
plt.grid(True)

# Model 2: 2D LDA (School Year + English_Level)
plt.subplot(1,2,2)
# Scatter plot
for label in [0,1]:
    plt.scatter(train_df['School Year'][y_train==label],
                train_df['English_Level'][y_train==label],
                label=f'Part Time Job={label}', s=100)

# LDA decision line
coef = model2.coef_[0]
intercept = model2.intercept_[0]
# Line: coef[0]*x1 + coef[1]*x2 + intercept = 0
# Solve for x2 (English_Level)
x_vals = np.array([train_df['School Year'].min()-0.5, train_d
f['School Year'].max()+0.5])
y_vals = -(coef[0]*x_vals + intercept)/coef[1]
plt.plot(x_vals, y_vals, 'r--', label='Decision boundary')
plt.xlabel('School Year')
plt.ylabel('English Level')
plt.title('Model 2: LDA Decision Boundary (2D)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```



Summary

Linear Discriminant Analysis. I have to be honest. It doesn't feel like the other models.

Most models you've seen so far:

- Try to **fit the data**
- Try to **optimize predictions directly**
- Try to **draw a boundary as best as they can**

They feel reactive. LDA is different. It doesn't rush to classify. It pauses. It studies the structure first. And that's probably why most instructors skip it (or instructors only briefly teach it, as it usually does not appear on exams).

Not because it's useless. But because it's uncomfortable.

It forces you to think in terms of: **Means** → **Variance** → **Scatter matrices** → **Projections**

Not just code. Not just `.fit()` and `.predict()`. And obviously the math can feel heavy. The steps can feel long. And the intuition doesn't click immediately.

But if you stay with it, something interesting happens. You start to see classification differently. Not as: "How do I separate these points?". But as: "What transformation makes this separation obvious?" → That shift? That's powerful.

Because LDA teaches you something deeper than just a model:

"Sometimes the problem is not hard... you're just looking at it from the wrong direction."

And once you find the right direction. The boundary draws itself.

References

<https://www.geeksforgeeks.org/machine-learning/ml-linear-discriminant-analysis/>

<https://www.ibm.com/think/topics/linear-discriminant-analysis>

<https://www.datacamp.com/tutorial/linear-discriminant-analysis>

<https://www.youtube.com/watch?v=azXCzI57Yfc>

Next Section:

 [25. Quadratic Discriminant Analysis](#)